

Gas Turbine Performance Modeling

Quick Start & User Guide

cy23 is a modular 0D gas turbine performance modeling framework supporting design and off-design analysis using component maps and a nonlinear solver architecture.

Table of Contents

Table of Contents 2
1. Overview 3
2. Preparing Component Maps 3
  2.1 Compressor Map Format 3
  2.2 Turbine Map Format 3
3. Building an Engine Model 3
4. Solver Setup and Execution 4
5. Performance Calculation 4
6. Writing Results 4
7. Batch Run 5
8. Model Persistence 5

1. Overview

The cy23 package enables users to:
• Construct engine models from modular components (compressor, turbine, burner, nozzle, etc.)
• Use Walsh & Fletcher thermodynamic relations
• Incorporate compressor and turbine maps for off-design performance
• Solve nonlinear matching problems using Independent and Dependent variables
• Compute performance metrics and export results to Excel

All user scripts require only a single import statement:

```
import cy23 as cy
```

2. Preparing Component Maps

Compressor and turbine maps are provided as CSV files and converted to pickled interpolators for fast runtime execution.

2.1 Compressor Map Format

Required CSV columns: NrT, Wc, PR, Eta

```
import cy23 as cy
cy.processCmap('.\\cmap1.csv')
```

This produces cmap1.pkl in the same directory.

2.2 Turbine Map Format

Required CSV columns: NrT, ER, Wc, Eta (optional Swirl)

```
cy.processTmap('.\\tmap1.csv')
```

This produces tmap1.pkl.

3. Building an Engine Model

Create a model and add components:

```
import cy23 as cy

eng = cy.Model('MyEngine')
amb = eng.add(cy.ambient, 'amb')
comp = eng.add(cy.compressor, 'comp', amb.outlet)
turb = eng.add(cy.turbine, 'turb', comp.outlet)
noz  = eng.add(cy.nozzle, 'noz', turb.outlet)
fe   = eng.add(cy.flowEnd, 'fe', noz.outlet)
```

Attach preprocessed maps:

```
comp.MAP = cy.loadmap('.\\cmap1.pkl')
turb.MAP = cy.loadmap('.\\tmap1.pkl')
```

Save the design model for reuse:

```
cy.saveModel(eng, 'exampleModel.pkl')
```

4. Solver Setup and Execution

Load a saved model:

```
eng = cy.loadModel('exampleModel.pkl')
```

Define Independent and Dependent variables:

```
flow = cy.Independent(eng.amb, 'W')
shaftERR = cy.Dependent(eng.lpshft, 'ERR', 0.0)
```

Create and execute a solver:

```
solver = cy.Solver(eng)
solver.offdesign = False
solver.indeps = [flow]
solver.deps = [shaftERR]

solver()
```

Solver.offdesign controls design versus off-design mode. The solver automatically propagates this state to all components.

If convergence issues arise, setting the flag Solver.diag=True will print out a “diag.txt” file with the solver dependent and independent values at each iteration. This is useful for troubleshooting as large dependent errors indicates what may be causing model divergence.

5. Performance Calculation

Users may define custom performance functions to compute thrust, TSFC, specific thrust, and efficiency metrics. These values are typically stored in eng.params and executed post-convergence via model.post.

```
def performance():
    FN = eng.noz.Fg - eng.amb.W * eng.amb.V
    eng.params.FN = FN

model.post = performance
```

A generic performance utility (cy.Performance) is also available for standard configurations. Currently only the “jet” version is implemented.

```
# Model performance function.

eng.post=cy.Performance(eng,"jet")
```

6. Writing Results

The most basic output will write a full state export to Excel for a single point:

```
cy.write(eng, 'outputFile')
```

Full state export for all cases (each case in one row):

```
crw=cy.CaseWriter(eng)
crw.update() # call after each solver run if we want data saved.
crw.write('caseOutput') # finally write output.
```

Parameter-only export (same as above, but selected parameters only):

```
prw = cy.ParamWriter(eng, ['amb.W', 'params.FN_lbf'])
prw.update()# call after each solver run if we want data saved.
prw.write('paramOutput') # finally write output.
```

7. Batch Run

The most useful utility when running multiple cases is to run a batch run. This allows an Excel spreadsheet to be used to specify inputs for multiple cases. The desired outputs must be specified (similar to ParamWriter above). When run, all results are written to the original spreadsheet input, to the right of the inputs, on a row by row basis for each input case. When running a batch run, both the Excel filename and tab name should be specified.

```
# First we define the outputs desired.

outputs=["shft.Nmech", "gen.kWe", "c1.outlet.Pt","c1.outlet.Tt",
"c2.outlet.Pt","c2.outlet.Tt""c3.outlet.Pt","c3.outlet.Tt","exh.inlet.T
t"]

# The BatchRun instance is declared specifying the solver, inputs and
outputs.

runTETs=cy.BatchRun(fuelMatch, ["amb.T", "amb.P", "comb.Wfuel_in"],
outputs)

# The batch run is run by calling the spreadsheet and tab name where
the input values are declared.

runTETs("calibModel.xlsx", "TET1750_wfactors")
```

There is also a MOPA (multiple operating point gas path analysis) for model calibration via map scalars. This is shown in provided examples.

8. Model Persistence

Model persistence stores component states and design scalars but does not store solver definitions or performance functions.

Save a model:

```
cy.saveModel(eng, 'model.pkl')
```

Load a model:

```
eng = cy.loadModel('model.pkl')
```

This is useful to store a model after sizing (running a “design-point” case) or saving converged model states to initialize other cases.